

the importance of adding a constraint to the alignment loss. **2)** L_A^{RDC2} , which applies a detached constraint solely to the lowest-quality examples, attains a marginally improved accuracy of 12.57. However, it also considerably impairs the model’s overall performance compared with VFT, indicating that constraining only the lowest-quality examples is insufficient. **3)** L_A^{RDC1} is much better than VFT, L_A^{RDC2} and $\mathcal{L}_A$, we think the reason is that after detaching all negative scores, L_A^{RDC1} prevents the model degradation, however, it is worse than L_A^{RBC} , we hypothesis that L_A^{RDC1} only tries to improve all scores, although with different extends, which is not good enough in the ranking situation.

D DELVE DEEPLY INTO PREVIOUS RANKING LOSSES FOR ALIGNMENT

In this section, we delve deeply into previous widely used ranking losses for alignment, DPO (Rafailov et al., 2023), RRHF (Yuan et al., 2023b) and PRO (Song et al., 2023a), and point out that they all suffer from lack of a constraint term.

Given a ranking sequence $c_1 \succeq c_2 \succeq \dots \succeq c_k$, all ranking losses are proposed to ensure the scores of high-quality examples are larger than those of low-quality examples. Ranking losses usually use the token-averaged log-likelihood to represent the score of an example c given by an LLM parameterized by θ :

$$s_\theta^c = \frac{1}{|c|} \sum_{j=1}^{|c|} \log P(c_j | c_{<j}, q; \theta), \quad (15)$$

D.1 DPO

Direct Preference Optimization (DPO) (the ranking version) optimizes LLMs with the following ranking loss:

$$\begin{aligned} \mathcal{L}_{DPO} &= - \sum_{c_i} \log \frac{\exp(\beta s_\theta^{c_i} - \beta s_{\theta_{ref}}^{c_i})}{\exp(\beta s_\theta^{c_i} - \beta s_{\theta_{ref}}^{c_i}) + \sum_{c_j \prec c_i} \exp(\beta s_\theta^{c_j} - \beta s_{\theta_{ref}}^{c_j})} \\ &= \sum_{c_i} \log \left[1 + \sum_{c_j \prec c_i} \exp(\beta s_\theta^{c_j} - \beta s_{\theta_{ref}}^{c_j} - \beta s_\theta^{c_i} + \beta s_{\theta_{ref}}^{c_i}) \right] \end{aligned} \quad (16)$$

where θ and θ_{ref} are parameters of the training model and reference model, respectively. The training model and reference model are usually initialized by the same LLM, and DPO freezes the reference model during fine-tuning. β is a hyper-parameter of DPO.

To analyze the effectiveness of DPO, we compute the gradient with respect to the parameters θ :

$$\begin{aligned} \nabla_\theta \mathcal{L}_{DPO} &= - \sum_{c_i} \frac{\sum_{c_j \prec c_i} [\beta \exp(\beta s_\theta^{c_j} - \beta s_{\theta_{ref}}^{c_j} - \beta s_\theta^{c_i} + \beta s_{\theta_{ref}}^{c_i}) \nabla_\theta s_\theta^{c_i} - \beta \exp(\beta s_\theta^{c_j} - \beta s_{\theta_{ref}}^{c_j} - \beta s_\theta^{c_i} + \beta s_{\theta_{ref}}^{c_i}) \nabla_\theta s_\theta^{c_j}]}{1 + \sum_{c_j \prec c_i} \exp(\beta s_\theta^{c_j} - \beta s_{\theta_{ref}}^{c_j} - \beta s_\theta^{c_i} + \beta s_{\theta_{ref}}^{c_i})} \end{aligned} \quad (17)$$

Based on $\nabla_\theta \mathcal{L}_{DPO}$, for each pair (c_i, c_j) , $\mathcal{L}_{DPO}$ will decrease the $s_\theta^{c_j}$ with the gradient weight $\frac{\beta \exp(\beta s_\theta^{c_j} - \beta s_{\theta_{ref}}^{c_j} - \beta s_\theta^{c_i} + \beta s_{\theta_{ref}}^{c_i})}{1 + \sum_{c_j \prec c_i} \exp(\beta s_\theta^{c_j} - \beta s_{\theta_{ref}}^{c_j} - \beta s_\theta^{c_i} + \beta s_{\theta_{ref}}^{c_i})}$, which may lead the model degradation.

In the original DPO paper (Rafailov et al., 2023), they observe this catastrophe and alleviate it by setting a very small β (e.g., 0.1) to achieve a small gradient weight. Please refer to the original paper for more details. However, based on Equation 17, the small β also hamper the improvement of positive examples, which may also hinder the model’s performance. Furthermore, solely relying on reducing gradient weights might not be sufficient to prevent model deterioration, as demonstrated in the subsequent analysis of RRHF and PRO. In this paper, we do not replicate DPO since there is no official public code available for ranking.

Scaling Factor β	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
Accuracy	18.75	18.01	15.05	13.20	11.79	11.79	9.83	8.78	8.62	7.51

Table 9: The influence of gradient weight scaling factor β for RRHF.

D.2 RRHF

Rank Responses to align Human Feedback (RRHF), which takes candidate ranking into account and distinguishes different candidates through a pair-wise ranking loss:

$$\mathcal{L}_{RRHF} = \sum_{c_i \succ c_j} \max(0, s_{\theta}^{c_j} - s_{\theta}^{c_i}) \quad (18)$$

We compute the gradient of $\mathcal{L}_{RRHF}$ with respect to θ :

$$\nabla_{\theta} \mathcal{L}_{RRHF} = - \sum_{c_i \succ c_j} \left[\underbrace{\mathbb{I}(s_{\theta}^{c_j} > s_{\theta}^{c_i}) \nabla_{\theta} s_{\theta}^{c_i}}_{\text{increase } s_{\theta}^{c_i}} - \underbrace{\mathbb{I}(s_{\theta}^{c_j} > s_{\theta}^{c_i}) \nabla_{\theta} s_{\theta}^{c_j}}_{\text{decrease } s_{\theta}^{c_j}} \right] \quad (19)$$

Based on $\nabla_{\theta} \mathcal{L}_{RRHF}$, we can see that although RRHF implicitly introduces a constraint by setting the loss to 0 when the positive score is larger than the negative score, it still has a drawback: Whenever $s_{\theta}^{c_j} > s_{\theta}^{c_i}$, $\mathcal{L}_{RRHF}$ will decrease the $s_{\theta}^{c_j}$ with the same gradient weight $\mathbb{I}(s_{\theta}^{c_j} > s_{\theta}^{c_i}) = 1$. This weight might be too large, potentially harming the model’s performance.

To illustrate this, we explore the performance of RRHF with a scaling factor β on its gradient weight. As shown in Table 9, it is evident that as the weight increases (larger β), the model’s performance declines, showing that: 1) The constraint of RRHF is not effective enough to prevent model degradation; 2) We can alleviate the model degradation by making the gradient weight smaller suggested by DPO (Rafailov et al., 2023); 3) Although we have tried a very small $\beta = 0.1$, RRHF still harms the performance, which shows solely relying on reducing gradient weights might not be sufficient to prevent model deterioration.

In fact, in the original RRHF paper (Yuan et al., 2023b), the authors have observed that a large ranking weight, such as 10 or 100, significantly impairs model performance, leading them to try a smaller weight (i.e., 1). However, they do not analyze the potential reason. In this paper, we highlight that a key factor causing this discrepancy is the unwarranted reduction of the negative example score, which necessitates imposing a constraint on the ranking loss. In addition, we discovered that a weight of 1 can also substantially harm the model’s performance in the reasoning task. We believe that the optimal weight of RRHF varies across tasks.

D.3 PRO

Preference Ranking Optimization (PRO), which takes candidate ranking into account and distinguishes different candidates through a ranking loss with a dynamic temperature:

$$\mathcal{L}_{PRO} = - \sum_{c_i} \log \frac{\exp(\tau_i^{max} s_{\theta}^{c_i})}{\exp(\tau_i^{max} s_{\theta}^{c_i}) + \sum_{c_j \prec c_i} \exp(\tau_i^j s_{\theta}^{c_j})} = \sum_{c_i} \log [1 + \sum_{c_j \prec c_i} \exp(\tau_i^j s_{\theta}^{c_j} - \tau_i^{max} s_{\theta}^{c_i})] \quad (20)$$

$$\tau_i^j = r^{c_i} - r^{c_j} > 0, \quad \tau_i^{max} = \max_{c_j \prec c_i} \tau_i^j \quad (21)$$

where r^c is the score of c given by a reward model. τ_i^j is the dynamic temperature for score $s_{\theta}^{c_j}$. We compute the gradient with respect to the parameters θ :

$$\nabla_{\theta} \mathcal{L}_{PRO} = - \sum_{c_i} \frac{\sum_{c_j \prec c_i} [\tau_i^{max} \exp(\tau_i^j s_{\theta}^{c_j} - \tau_i^{max} s_{\theta}^{c_i}) \nabla_{\theta} s_{\theta}^{c_i} - \tau_i^j \exp(\tau_i^j s_{\theta}^{c_j} - \tau_i^{max} s_{\theta}^{c_i}) \nabla_{\theta} s_{\theta}^{c_j}]}{1 + \sum_{c_j \prec c_i} \exp(\tau_i^j s_{\theta}^{c_j} - \tau_i^{max} s_{\theta}^{c_i})} \quad (22)$$

Methods	PRO	PRO (remove τ)	PRO + RDC1	PRO (remove τ) + RDC1
Accuracy	18.73 $\pm$ 0.31	7.18 $\pm$ 0.78	25.84 $\pm$ 0.48	25.43 $\pm$ 0.98

Table 10: The importance of dynamic temperature of PRO. “remove τ ” denotes remove the dynamic temperature term, i.e., τ_j^j and τ_i^{max} from PRO. “+RDC1” denotes add our ranking detach technical (Equation 12).

Based on $\nabla_{\theta} \mathcal{L}_{PRO}$, we can see that for each pair (c_i, c_j) , $\mathcal{L}_{PRO}$ will decrease $s_{\theta}^{c_j}$ with the dynamic gradient weight:

$$DGW_i^j = \frac{\tau_i^j \exp(\tau_i^j s_{\theta}^{c_j} - \tau_i^{max} s_{\theta}^{c_i})}{1 + \sum_{c_j < c_i} \exp(\tau_i^j s_{\theta}^{c_j} - \tau_i^{max} s_{\theta}^{c_i})}, \quad (23)$$

which may harm the model’s performance. However, the dynamic gradient weight that is computed based on the reward is more reasonable than the constant value of 1 used in RRHF, and thus PRO outperforms RRHF. Specifically, when there is a substantial reward gap between higher-quality and lower-quality, indicated by a large value τ_i^j . It is reasonable to increase the penalty for negative example scores (large DGW_i^j), and vice versa. To demonstrate this, we remove the dynamic temperature term, i.e., τ_j^j and τ_i^{max} , from PRO. As shown in Table 10, we can see that PRO significantly outperforms PRO (remove τ) when there is no constraint. However, the performance gap shrinks when adding our detached constraint. These results indicate: 1) To a certain extent, the dynamic temperature’s effectiveness stems from its ability to make PRO reduce the negative score in a more reasonable manner. 2) The dynamic temperature is useful to prevent model degradation but is not good enough.

D.4 SUMMARY

Our analysis reveals that previous ranking-based alignment works have some limitations:

- 1) Although their methods consist of some strategies to prevent model degradation (i.e., using a scaling factor to reduce gradient weight for DPO, setting the loss to 0 for RRHF, and incorporating a dynamic temperature for PRO), they do not recognize the importance of constraints for ranking-based alignment methods in their papers.
- 2) Their strategies essentially involve diminishing the gradient weight’s magnitude, which is proven to be insufficient (at least in the reasoning tasks).

Different from previous works, in this paper:

- 1) We point out an assessment misalignment problem of VFT for reasoning and highlight the importance of constraint for alignment to prevent model degradation.
- 2) We introduce a novel constraint alignment loss. The constraint alignment loss with a boundary constraint term not only alters the magnitude but also adjusts the direction of the gradient weight depending on the condition, which is proven to be very effective in preventing model degradation and enhancing the reasoning ability of LLMs.

E CASE STUDY

We also conducted a case study to intuitively show the importance of our constraint alignment. As shown in Table 11, given the question, our AFT successfully gives the correct COT and answer, while VFT gives the wrong COT at the second step (colored red), demonstrating the superiority of AFT. More importantly, after removing the boundary constraint, the generative ability of LLM seems to degrade, resulting in outputting many repeat and meaningless output tokens.